

DEVELOPER GUIDE DG-001

A TRAVELER'S GUIDE TO CODE-BASES

FOR OFFICIAL USE ONLY

A TRAVELER'S GUIDE TO CODEBASES

NAVIGATING FOREIGN TERRITORY

BEFORE YOU ARRIVE

Pack Your Tools

- A good IDE with search and navigation
- Git client for version control

- Documentation reader (browser, PDF viewer)
- Terminal for running commands
- Debugger for exploring

Study the Map

- README.md - Your starting point
 - Architecture docs - The big picture
 - API documentation - How things connect
 - Test files - Examples of how things work
-

WHEN YOU ARRIVE

Establish Base Camp

1. Clone the repository
2. Read the README
3. Set up the development environment
4. Run the tests (they should pass)
5. Start the application (it should work)

Explore the Neighborhood

- Find the entry point (main.py, index.js, etc.)
 - Identify the core modules
 - Map the data flow
 - Understand the dependencies
-

COMMON LANDMARKS

The Entry Point

Where it all begins. Usually named `main`, `index`, or `app`. This is your starting point.

The Configuration

Settings, environment variables, config files. Know where these live—they control behavior.

The Data Layer

Models, schemas, database connections. This is where data lives and moves.

The Business Logic

The core functionality. This is what the application actually does.

The Interface Layer

APIs, UI, CLI. This is how users (and other systems) interact with your code.

NAVIGATING CHALLENGES

When You're Lost

1. Use search to find related code
2. Follow the call stack
3. Read the tests—they show expected behavior
4. Ask questions (in code comments, docs, or to teammates)

When You Need to Change Something

1. Find all the places it's used (search is your friend)
2. Understand the current behavior
3. Write a test for the new behavior
4. Make the change
5. Verify tests still pass
6. Update documentation

When You Find Something Strange

1. Don't assume it's wrong—there might be a reason
 2. Check git history—see why it was added
 3. Look for comments or documentation
 4. Ask the original author (if possible)
-

LEAVING YOUR MARK

When You Add Features

- Follow existing patterns
- Write tests
- Update documentation
- Keep it simple

When You Refactor

- Don't change behavior (unless that's the goal)
 - Update tests if needed
 - Document why you changed it
 - Make it better, not just different
-

PARTING WISDOM

Every codebase is a journey. Take your time. Ask questions. Learn the landscape. And remember: you're not just visiting—you're contributing to a living, evolving system.

Happy travels!